CAMPUSCARE: HOSTEL AND FACILITY COMPLAINT MANAGEMENT SYSTEM

A PROJECT REPORT

Submitted by

SUBHALAKSHMI M

(240701539)

in partial fulfilment of the End Semester Examination for the course

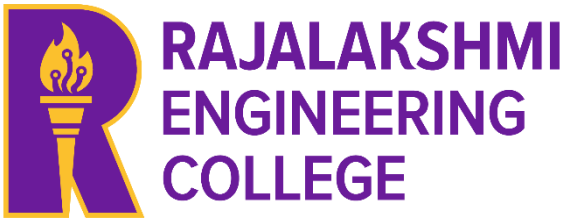
CB23F35 –

LOW-CODE APPLICATION DEVELOPMENT USING ZOHO CREATOR

in

BACHELOR OF ENGINEERING

COMPUTER SCIENCE AND ENGINEERING



RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

Rajalakshmi Nagar, Thandalam, Tamil Nadu – 602 105

NOVEMBER 2026

RAJALAKSHMI ENGINEERING COLLEGE

(An Autonomous Institution, Affiliated to Anna University, Chennai)

BONAFIDE CERTIFICATE

Certified that this project report titled "CAMPUSCARE: HOSTEL AND FACILITY COMPLAINT MANAGEMENT SYSTEM" is the bonafide work of SUBHALAKSHMI M (240701539), who carried out the individual project work on the Zoho Creator low-code platform for the course CB23F35 – Low-Code Application Development using Zoho Creator, during the academic year 2026–2027, Semester V, under my supervision.

SIGNATURE

Dr. J. Manoranjini
HEAD OF THE DEPARTMENT
Department of Computer Science and Engineering
Rajalakshmi Engineering College
Thandalam – 602 105

SIGNATURE

Dr. J. Jinu Sophia
COURSE FACULTY
Associate Professor
Department of Computer Science and Engineering
Rajalakshmi Engineering College

Submitted for the End Semester Examination (Individual Project) held on 11 November 2026.

INTERNAL EXAMINER

EXTERNAL EXAMINER

DECLARATION

I, SUBHALAKSHMI M (240701539), hereby declare that the project report titled "CAMPUSCARE: HOSTEL AND FACILITY COMPLAINT MANAGEMENT SYSTEM", submitted for the End Semester Examination of the course CB23F35 – Low-Code Application Development using Zoho Creator, is a record of original individual project work. The application design, data model, forms, reports, workflows, Blueprint, portal configuration and Deluge scripts documented in the final report will reflect the work completed by me. Any templates, platform documentation, sample resources or AI-assisted tools used during development will be acknowledged appropriately.

I further declare that this work has not been submitted, in part or in full, for the award of any other degree or course credit. The final report will distinguish implemented features from proposed enhancements and will include genuine screenshots, test evidence and source references.

Place: Chennai
Date: 11 November 2026

SIGNATURE OF THE STUDENT
SUBHALAKSHMI M
240701539

ACKNOWLEDGEMENT

I express my sincere gratitude to our Chairman Mr. S. Meganathan, B.E., F.I.E., Chairperson Dr. Thangam Meganathan, Ph.D., and Vice-Chairman Mr. Abhay Shankar Meganathan, B.E., M.S., for providing the infrastructure and facilities that made this work possible.

I am grateful to our Principal, Dr. S.N. Murugesan, M.E., Ph.D., Rajalakshmi Engineering College, for the constant encouragement extended to students to take up industry-relevant learning.

I thank Dr. J. Manoranjini, Professor and Head of the Department, Department of Computer Science and Engineering, for the guidance and support provided throughout the course.

I owe my sincere thanks to my course faculty, Dr. J. Jinu Sophia, Associate Professor, for the valuable guidance, reviews and feedback that shaped this application and report.

I also acknowledge Zoho Corporation and the Zoho Creator Education Programme for providing access to the Zoho Creator platform and learning resources, and I thank my family and friends for their unwavering support.

SUBHALAKSHMI M

ABSTRACT

Hostel residents in a college raise a steady stream of maintenance and facility complaints about water supply, electricity, Wi-Fi, cleaning and furniture. In many institutions these complaints are still passed on verbally, noted in paper registers or tracked in spreadsheets. As a result the warden has no single view of what is pending, the right maintenance person is found through phone calls, students do not know the status of their request, and no usable history exists for review.

CampusCare: Hostel and Facility Complaint Management System was developed on Zoho Creator to replace this manual process. A low-code approach was chosen because it allowed a working, hosted and mobile-friendly application to be delivered quickly without separate infrastructure. The application has five forms (Complaint, Student Details, Staff Details, Work Log and Feedback) connected through lookup fields, five list reports and three kanban reports, and a dashboard page with key counts and charts. A Deluge workflow, triggered on successful submission of a complaint, fetches an available staff member whose specialization matches the complaint category, assigns the complaint, updates its status and sends email notifications to the staff member and the student. A role-based access model for the warden, maintenance staff and students is defined for the application.

The application was tested with sample student, staff, complaint, work-log and feedback records. Defects found during development, such as script references to forms and fields that did not yet exist, were identified through the error messages of the Deluge editor and corrected. Test results show that complaints are registered with a system-generated identifier, grouped by category, logged against staff actions and summarised on the dashboard, which gives the warden a single, up-to-date view of all hostel complaints. The application removes repeated manual entry, makes the status visible to students and keeps a permanent record of the work done.

Keywords: *Low-Code Development; Zoho Creator; Deluge Scripting; Workflow Automation; Role-Based Access Control; Hostel Complaint Management*

PROJECT AT A GLANCE

Attribute	Details
Project Title	CampusCare: Hostel and Facility Complaint Management System
Application Domain	Education – college hostel facility management
Student Name & Register No.	Subhalakshmi M – 240701539
Zoho Creator Edition	Trial edition
Application Name (Link Name)	Hostel Facility Complaint System
Published / Live URL	https://creatorapp.zoho.in/subha2k07/hostel-complaint-management-system/#Form:complaint
Demo Credentials (per role)	Student/wardern/staff
Number of Forms / Subforms	5 / 0
Number of Reports (by type)	List: 5 Kanban: 3 Calendar: 0 Pivot/Chart: 2 (dashboard charts) Others: 0
Number of Pages / Dashboards	1
Number of Workflows	Form: 1 Schedule: 0 Approval: 0 Blueprint: 0 Batch: 0
Deluge Functions / Custom Actions	1 form-event script (DS-01)
Integrations	Zoho Mail (sendmail task)
Roles & Profiles Defined	Administrator / Warden, Maintenance Staff, Student
Environments Used	Development (live-mode testing)
Date of Production Deployment	02-11-2026

TABLE OF CONTENTS

ABSTRACT ii
PROJECT AT A GLANCE iii
LIST OF TABLES vii
LIST OF FIGURES viii
LIST OF ABBREVIATIONS ix
INTRODUCTION 1
1.1 Background 1
1.2 Problem Statement 1
1.3 Objectives 2
1.4 Scope of the Project 2
1.5 Justification for a Low-Code Approach 2
1.6 Organisation of the Report 3
REVIEW OF EXISTING SYSTEM AND RELATED WORK 4
2.1 Existing System 4
2.2 Related Low-Code Work 4
2.3 Gap Identified 4
REQUIREMENT ANALYSIS 5
3.1 Stakeholders and User Roles 5
3.2 Functional Requirements 5
3.3 Non-Functional Requirements 6
3.4 Hardware and Software Requirements 6
3.5 Use Case Model 7
SYSTEM DESIGN 9
4.1 System Architecture 9
4.2 Data Model 9
4.3 Form Design 10
4.4 Process and Workflow Design 13
4.5 User Interface and Navigation Design 14
4.6 Access Control Design 15
IMPLEMENTATION 17
5.1 Application Setup 17
5.2 Forms and Data Capture 17
5.3 Reports 20
5.4 Pages and Dashboards 22
5.5 Deluge Scripting 22
5.6 Workflows and Automation 23
5.7 Integrations 24

5.8 User Management and Role-Based Access 25
5.9 AI and Analytics Features 25
5.10 Pro-Code Extension 26
5.11 Mobile and Portal Access 26
TESTING AND DEBUGGING 27
6.1 Testing Strategy 27
6.2 Test Cases 27
6.3 Debugging Deluge Scripts 28
6.4 Defect Log 29
6.5 User Acceptance Testing 30
DEPLOYMENT, SECURITY AND GOVERNANCE 31
7.1 Environments and Deployment Strategy 31
7.2 Performance Optimisation 31
7.3 Security Features 32
7.4 Governance and Administration 32
RESULTS AND DISCUSSION 34
8.1 Application Output 34
8.2 Achievement of Objectives 35
8.3 Comparison with the Existing System 35
8.4 Limitations of the Low-Code Approach 36
8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum 36
CONCLUSION AND FUTURE ENHANCEMENTS 37
9.1 Conclusion 37
9.2 Future Enhancements 37
9.3 Learning Outcomes and Career Relevance 37
REFERENCES 38
APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST 39
APPENDIX B – COURSE OUTCOME MAPPING 41
APPENDIX C – COMPLETE DELUGE SOURCE CODE 42
APPENDIX D – USER MANUAL 43
D.1 Accessing the Application 43
D.2 Instructions for the Student 43
D.3 Instructions for the Maintenance Staff 43
D.4 Instructions for the Warden / Administrator 43
APPENDIX E – EVALUATION RUBRIC 44
APPENDIX F – VIVA-VOCE RECORD SHEET 46

LIST OF TABLES

Table 3.1: Stakeholders and User Roles 5
Table 3.2: Functional Requirements 5
Table 3.3: Non-Functional Requirements 6
Table 3.4: Hardware and Software Requirements 6
Table 3.5: Use Case Description – Submit Complaint 7
Table 3.6: Use Case Description – Update Work Log and Resolve Complaint 8
Table 4.1: Entities and Relationships 10
Table 4.2: Field Specification – Complaint Form 11
Table 4.3: Field Specification – Student_Details Form 11
Table 4.4: Field Specification – Staff_Details Form 12
Table 4.5: Field Specification – Work_Log Form 12
Table 4.6: Field Specification – Feedback Form 13
Table 4.7: Role–Permission Matrix 15
Table 5.1: Reports Implemented 20
Table 5.2: Deluge Scripts Implemented 22
Table 5.3: Workflows Implemented 24
Table 5.4: Integrations Implemented 25
Table 6.1: Test Cases 27
Table 6.2: Defect Log 29
Table 6.3: User Acceptance Feedback 30
Table 7.1: Deployment Checklist 31
Table 7.2: Performance Optimisation Results 32
Table 7.3: Security Controls Implemented 32
Table 8.1: Mapping of Objectives to Outcomes 35
Table 8.2: Existing System versus Proposed Application 35
Table A.1: Feature Coverage Checklist 39
Table B.1: Mapping of Course Outcomes to Report Chapters 41
Table E.1: Evaluation Rubric for the Individual Project 44
Table E.2: Performance Level Descriptors 45
Table F.1: Viva-Voce Questions and Responses 46
Table F.2: Marks Summary 46

LIST OF FIGURES

Figure 3.1: Use Case Diagram of the Application 7
Figure 4.1: System Architecture of the Application 9
Figure 4.2: Entity–Relationship Diagram 10
Figure 4.3: Process Flow for Complaint Registration and Resolution 14
Figure 4.4: Navigation Structure of the Application 15
Figure 5.1: Application Home and Navigation Sections 17
Figure 5.2: Complaint Form in Live Mode 18
Figure 5.3: Lookup Field Configuration for Assigned Staff 18
Figure 5.4: Complaint Form with the Quick-Add Staff Details Window 19
Figure 5.5: Work Log Form in Live Mode 19
Figure 5.6: All Complaints Report 20
Figure 5.7: Categories Kanban Report 21
Figure 5.8: Work Log Report 21
Figure 5.9: All Student Details Report 21
Figure 5.10: Departments Kanban Report 21
Figure 5.11: Staff Details Report 21
Figure 5.12: All Feedback Report 21
Figure 5.13: Dashboard Charts and Embedded All Complaints Report 22
Figure 5.14: Creating the Workflow on the Complaint Form 24
Figure 5.15: Workflow Created on Successful Form Submission and Action Type Selection 24
Figure 5.16: Roles and Permission Profiles Configuration 25
Figure 6.1: Deluge Editor Error – Form Not Found 29
Figure 6.2: Deluge Editor Error – Field Not Found 29
Figure 7.1: Environments / Publish to Production 31
Figure 7.2: Audit Trail / Administration Settings 33
Figure 8.1: Scenario Step 1 – Student Registers a Complaint 34
Figure 8.2: Scenario Step 2 – Complaint Appears in the All Complaints Report 34
Figure 8.3: Scenario Step 3 – Staff Records the Action in the Work Log 35
Figure 8.4: Scenario Step 4 – Work Log Report Shows the Entry 35

CHAPTER 1
INTRODUCTION

1.1 Background

A residential college hostel houses hundreds of students across several blocks (Block A to Block D in this project). Daily life in a hostel depends on basic facilities: water supply, electricity, Wi-Fi, cleaning and furniture. Whenever one of these fails, the student must report it to the warden or a caretaker, and someone from the maintenance team must be found to fix it.

In most colleges this process is still informal. A student tells the warden in person, writes the problem in a paper register or sends a message on a phone. The warden then calls the technician who is expected to be available. Three groups of people are involved: the students who raise complaints, the warden who monitors them, and the maintenance staff who repair the fault. None of them has a shared, up-to-date record of what has been reported, who is handling it and whether it is finished.

This project, **CampusCare: Hostel and Facility Complaint Management System**, builds that shared record as a web application on the Zoho Creator low-code platform. It is designed for a college hostel with students from several departments living in different blocks and rooms.

1.2 Problem Statement

Hostel complaints are reported and followed up through verbal requests, paper registers and phone calls. Because there is no single system, complaints are delayed or forgotten, the warden cannot see the overall position, students cannot find out the status of their request, and there is no history for reviewing the performance of maintenance work. The main pain points are:

- No central record: complaint details are scattered across registers and messages, and are often repeated or lost.
- Slow assignment: the warden has to find a suitable and available technician manually for each complaint.
- No visibility for students: a student cannot check whether the complaint was received, assigned or resolved.

- No reporting: complaints cannot be grouped by category, hostel block or priority to identify recurring problems.
- No accountability: the work done by maintenance staff is not recorded, so repeated visits and delays cannot be reviewed.

1.3 Objectives

- To design a normalised data model and forms for students, maintenance staff, complaints, work logs and feedback.
- To automate the assignment of complaints and the sending of notifications using Deluge scripting and a form workflow.
- To provide list, kanban and chart reports and a dashboard so that the warden can monitor complaints, staff and students.
- To define role-based access for students, the warden and maintenance staff.
- To test the application, record and fix defects, and document its deployment, security and governance aspects.

1.4 Scope of the Project

In scope: registration of complaints by category, priority, hostel block and room; student and staff master records; automatic assignment of an available staff member by specialization; email notification to the student and the assigned staff member; work log entries for each complaint; feedback with rating and comments; list and kanban reports; a dashboard with key counts and charts; and a role-based access model for the Administrator/Warden, Maintenance Staff and Student roles. The application is delivered as a web application and can be previewed in desktop, tablet and mobile layouts.

Out of scope: payment or purchase of spare parts, integration with biometric or hostel entry systems, SMS or WhatsApp gateways, and complaints for the academic buildings outside the hostel.

1.5 Justification for a Low-Code Approach

A hostel complaint system needs forms, lists, a few automatic actions and a dashboard. These are exactly the building blocks that a low-code platform provides out of the box. Low-code development replaces most hand-written code with visual configuration, and uses scripting only where business logic cannot be configured [1]. Zoho Creator was

chosen because it offers drag-and-drop forms, report types such as list and kanban, a page builder for dashboards, and the Deluge scripting language for workflow logic [2], [3].

The approach suits this problem for the following reasons. Development speed is high, since a working application with data entry, reports and automation could be assembled in days rather than weeks. Maintainability is good, because a warden or an administrator with no programming background can add a hostel block or a category by editing a drop-down. Hosting, backup and security are provided by the platform, so no server has to be managed. The application works in a browser and on mobile devices without separate development, and the cost of a trial or low-tier subscription is small compared with custom development.

1.6 Organisation of the Report

Chapter 2 reviews the existing system and related work. Chapter 3 presents the requirement analysis. Chapter 4 describes the system design. Chapter 5 details the implementation on Zoho Creator. Chapter 6 covers testing and debugging. Chapter 7 discusses deployment, performance optimisation, security and governance. Chapter 8 presents results and discussion, and Chapter 9 concludes the report with future enhancements.

CHAPTER 2
REVIEW OF EXISTING SYSTEM AND RELATED WORK

2.1 Existing System

In the present arrangement a student reports a hostel problem verbally to the warden or caretaker, or writes it in a complaint register kept at the hostel office. The warden decides who should attend to it and informs that person by phone. When the work is completed, the register entry is ticked or the student is told in person. Some hostels also use informal messaging groups, but the messages are mixed with other conversation and cannot be searched or summarised.

This arrangement has clear weaknesses: details have to be written more than once, there is no automatic reminder, the status can only be known by asking, and the warden has to count register entries by hand to prepare any report for the management.

2.2 Related Low-Code Work

Low-code application platforms allow business applications to be built through visual modelling of data, screens and processes, with scripting reserved for custom logic [1]. The Zoho Creator documentation describes forms, lookup relationships, reports, pages and workflows as the core building blocks of an application [2], and the Deluge language as the scripting layer that connects them, for example through fetch statements, conditional logic and the sendmail task [3]. The Zoho Creator learning resources provide sample applications for requests, approvals and tracking, which follow the same pattern of capture, assign, act and report that a complaint system needs [4].

2.3 Gap Identified

The manual system lacks a central record, automatic assignment, status tracking and reporting. The available low-code building blocks can supply all four. This project therefore applies them to a specific need: a hostel complaint application in which each complaint is registered once, assigned automatically where possible, logged during the repair and summarised for the warden.

CHAPTER 3
REQUIREMENT ANALYSIS

3.1 Stakeholders and User Roles

Table 3.1: Stakeholders and User Roles

Role	Description	Key Responsibilities in the Application
Administrator / Warden	Person in charge of the hostel; owner of the application	Maintain student and staff records, monitor complaints and the dashboard, assign complaints manually when needed, review feedback
Maintenance Staff	Plumbers, electricians, network technicians, cleaners and carpenters	View assigned complaints, record actions in the Work Log, mark complaints as resolved
Student (Hostel Resident)	Student living in a hostel block	Register complaints, check status, give feedback after resolution

3.2 Functional Requirements

Table 3.2: Functional Requirements

FR ID	Requirement	Role(s)	Priority
FR-01	The system shall allow a complaint to be registered with a title, description, category, priority, hostel block and room number.	Student, Warden	High
FR-02	The system shall generate a unique Complaint ID and record the submission date and time.	System	High
FR-03	The system shall maintain student records with register number, department, year, hostel block, room number and linked user account.	Warden	High
FR-04	The system shall maintain maintenance staff records with staff code, contact details, specialization and availability.	Warden	High
FR-05	The system shall automatically assign an available staff member whose specialization matches the complaint category and set the status to Assigned.	System	High
FR-06	The system shall send a confirmation email to the student and an assignment email to the staff member.	System	Medium
FR-07	The system shall allow staff to record each action taken on a complaint in a Work Log.	Maintenance Staff	High
FR-08	The system shall track the complaint status (Open, Assigned, In Progress, Resolved), the resolved date and the days pending.	All	High

FR ID	Requirement	Role(s)	Priority
FR-09	The system shall collect a rating and comments from the student for a complaint.	Student	Medium
FR-10	The system shall provide list and kanban reports of complaints, work logs, students, staff and feedback.	Warden, Staff	Medium
FR-11	The system shall display a dashboard of key counts and charts by priority, department and specialization.	Warden	Medium
FR-12	The system shall restrict data access according to the user role.	All	High

3.3 Non-Functional Requirements

Table 3.3: Non-Functional Requirements

NFR ID	Category	Requirement
NFR-01	Performance	List reports with up to 1,000 records shall load within a few seconds on a normal broadband connection.
NFR-02	Security	Users shall access only the data permitted by their role.
NFR-03	Usability	The application shall be usable in a web browser and on the Zoho Creator mobile app, with forms that can be completed in under two minutes.
NFR-04	Availability	The application shall be hosted on the Zoho cloud and be available whenever the platform is available.
NFR-05	Maintainability	Categories, hostel blocks and departments shall be changeable through drop-down choices without writing code.
NFR-06	Auditability	Changes to complaint records shall be traceable through the platform audit trail where supported.

3.4 Hardware and Software Requirements

Table 3.4: Hardware and Software Requirements

Category	Specification
Platform	Zoho Creator (trial edition used for development)
Scripting language	Deluge (Data Enriched Language for the Universal Grid Environment)
Browser	Google Chrome / Microsoft Edge / Mozilla Firefox (latest version)
API testing tool	Postman (not required, as no external API is called)
Other Zoho services	Zoho Mail (through the Deluge sendmail task)
Mobile	Zoho Creator mobile app (Android / iOS) and the tablet and mobile previews of the builder
Client hardware	Intel Core i3/i5 or above, 8 GB RAM, internet connection

3.5 Use Case Model

Figure 3.1 shows the use cases of the three roles. The student submits complaints, checks their status and gives feedback. The warden manages records, monitors complaints and views the dashboard. Maintenance staff view their assigned complaints, update the work log and mark complaints as resolved. Zoho Mail appears as a supporting system because it delivers the notification emails.

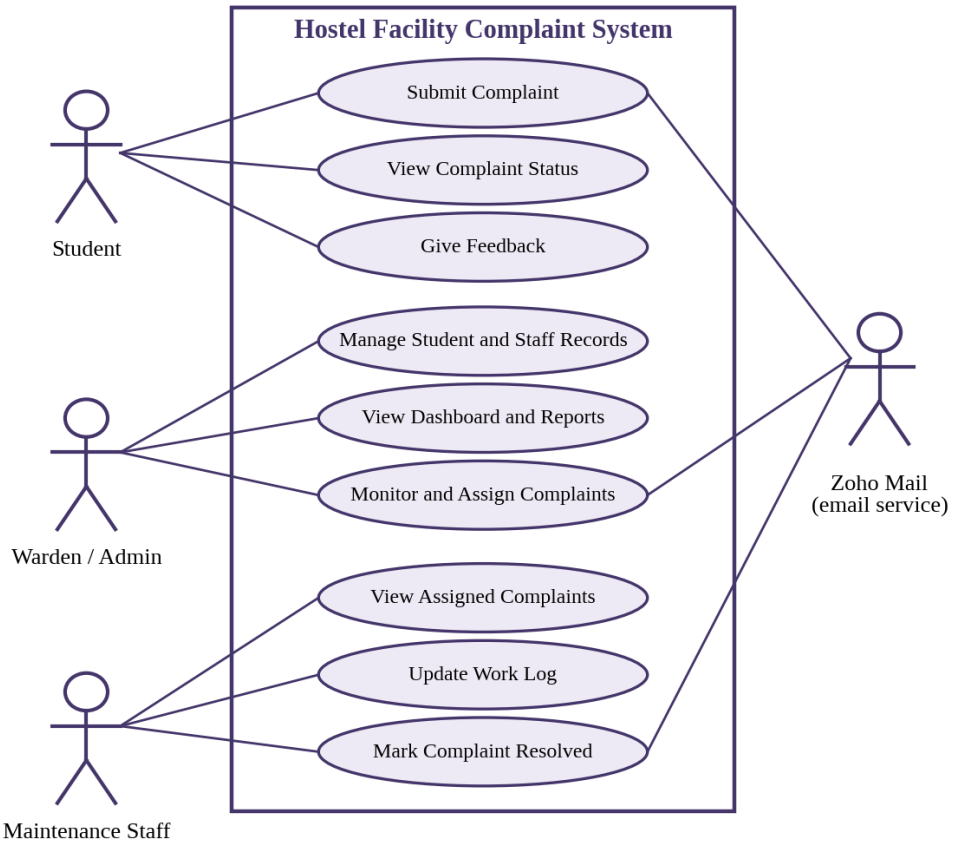


Figure 3.1: Use Case Diagram of the Application

Table 3.5: Use Case Description – Submit Complaint

Use Case ID	UC-01
Use Case Name	Submit Complaint
Actor(s)	Student (primary), Zoho Mail (supporting)
Pre-condition	The student has a record in Student_Details and can open the application.

Use Case ID	UC-01
Main Flow	1. The student opens Complaints > Complaint. 2. The student enters the title and description and selects category, priority, hostel block, room number and student. 3. The student clicks Submit. 4. The system validates the data and saves the record with a Complaint ID. 5. The workflow fetches an available staff member for the category, assigns the complaint, sets the status to Assigned and sends emails.
Alternate Flow	If a required value is missing, a validation message is shown and the record is not saved. If no available staff member matches the category, the complaint stays Open and the warden assigns it manually.
Post-condition	The complaint is visible in All Complaints and in the Categories kanban report.
Related FR	FR-01, FR-02, FR-05, FR-06

Table 3.6: Use Case Description – Update Work Log and Resolve Complaint

Use Case ID	UC-02
Use Case Name	Update Work Log and Resolve Complaint
Actor(s)	Maintenance Staff, Warden
Pre-condition	A complaint exists and a staff member has been assigned to it.
Main Flow	1. The staff member opens Work Log > Work Log. 2. The staff member selects the complaint, enters the log date and the action taken, and selects the staff member. 3. The entry is saved and appears in the Work Log Report. 4. When the repair is complete the status of the complaint is changed to Resolved and the resolved date is entered. 5. The student is invited to give feedback.
Alternate Flow	If the repair needs more than one visit, further Work Log entries are added and the status remains In Progress.
Post-condition	The complaint history shows every action taken; the dashboard counts are updated.
Related FR	FR-07, FR-08, FR-09

CHAPTER 4
SYSTEM DESIGN

4.1 System Architecture

The application follows the layered structure of a Zoho Creator solution, shown in Figure 4.1. The top layer contains the users and the channel through which they reach the application: a web browser or the Zoho Creator mobile app. The second layer is the application itself, made of forms for data capture, reports for viewing data, a dashboard page, a Deluge workflow, validation rules and the users-and-permissions model. All data is stored in the third layer, the Zoho Creator cloud data store, where relationships between forms are created through lookup fields. The fourth layer holds the services used by the application: Zoho Mail delivers notification emails through the sendmail task, and the Zia AI-assisted builder was used to create the initial structure. Further services such as Zoho Cliq, SMS or Zoho Analytics can be added later.

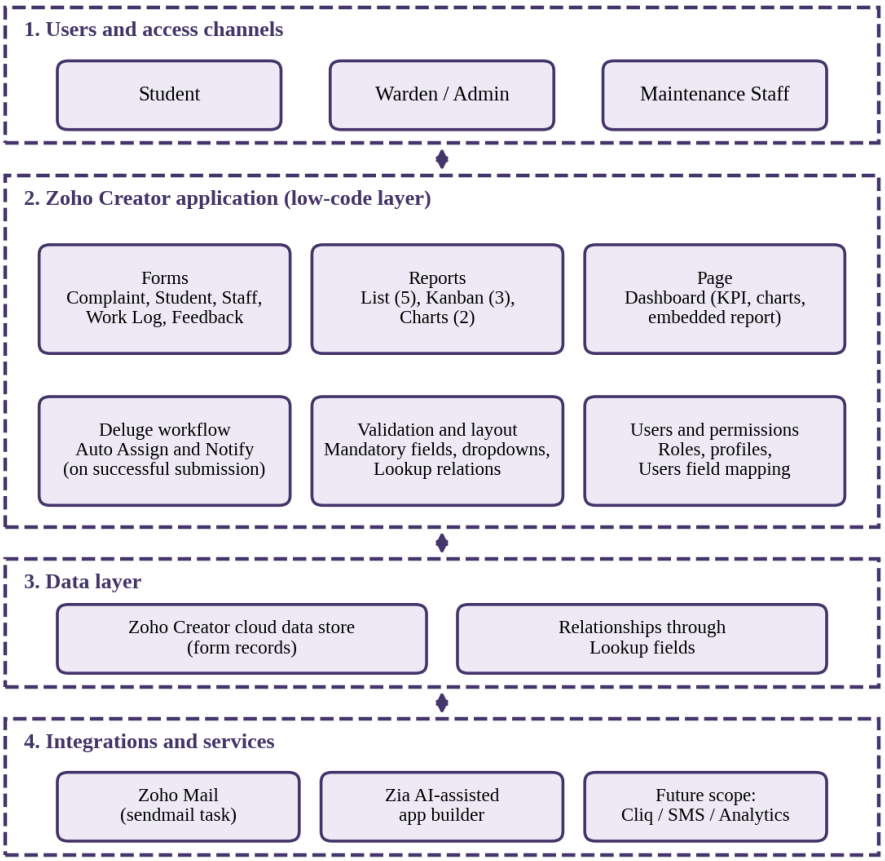


Figure 4.1: System Architecture of the Application

4.2 Data Model

The data model has five entities (Figure 4.2). A student can raise many complaints and a staff member can be assigned many complaints, so Complaint holds two lookup fields, one to Student_Details and one to Staff_Details. Each complaint can have many work-log entries and many feedback entries, so Work_Log and Feedback each hold a lookup to Complaint. Work_Log also holds a lookup to Staff_Details to record who carried out the work. All relationships are one-to-many and are implemented with lookup fields. A subform was not used for the work log, because work-log entries needed their own list report that could be filtered by staff member and complaint.

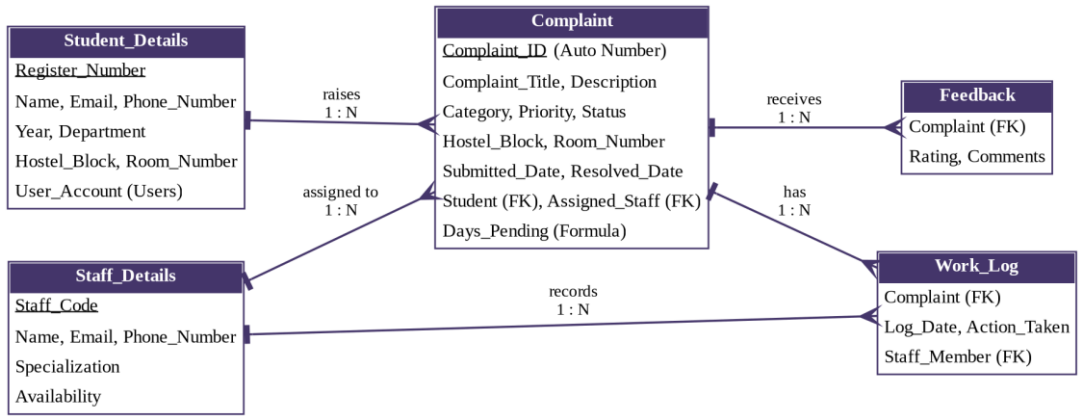


Figure 4.2: Entity–Relationship Diagram

Table 4.1: Entities and Relationships

Entity (Form)	Key Fields	Related To	Relationship Type	Implemented Using
Student_Details	Register Number, Name, Email, Department, Hostel Block	Complaint	One-to-Many	Lookup field Student in Complaint
Staff_Details	Staff Code, Name, Specialization, Availability	Complaint	One-to-Many	Lookup field Assigned Staff in Complaint
Complaint	Complaint ID, Title, Category, Priority, Status	Work_Log	One-to-Many	Lookup field Complaint in Work_Log
Staff_Details	Staff Code, Name	Work_Log	One-to-Many	Lookup field Staff Member in Work_Log
Complaint	Complaint ID, Status	Feedback	One-to-Many	Lookup field Complaint in Feedback

4.3 Form Design

Each form is specified below with the field label, the link name used in Deluge scripts, the Zoho Creator field type, whether the field is mandatory and any validation or default value.

Table 4.2: Field Specification – Complaint Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Complaint ID	Complaint_ID	Auto Number	—	System generated	Unique per complaint
Complaint Title	Complaint_Title	Single Line	Yes	—	Short summary
Description	Description	Multi Line	Yes	—	Used in staff email
Category	Category	Drop Down	Yes	Water, Electricity, Cleaning, Furniture, WiFi, Other	Drives auto-assignment
Priority	Priority	Drop Down	Yes	High, Medium, Low	Shown on dashboard
Hostel Block	Hostel_Block	Drop Down	Yes	Block A to Block D	
Room Number	Room_Number	Single Line	Yes	—	
Status	Status	Drop Down	Yes	Open, Assigned, In Progress, Resolved	Set to Assigned by workflow
Submitted Date	Submitted_Date	Date-Time	No	dd-MMM-yyyy HH:mm:ss	
Resolved Date	Resolved_Date	Date-Time	No	dd-MMM-yyyy HH:mm:ss	Filled on resolution
Attachment	Attachment	Multi Line	No	—	Reference to supporting file
Rating	Rating	Number	No	—	Entered after resolution
Signature	Signature	Multi Line	No	—	Name of the student
Student	Student	Lookup	Yes	Student_Details (Register Number)	Relationship 1
Assigned Staff	Assigned_Staff	Lookup	No	Staff_Details (Staff Code)	Set by workflow or warden
Days Pending	Days_Pending	Formula	—	Days since submission	Calculated field

Table 4.3: Field Specification – Student_Details Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Register Number	Register_Number	Single Line	Yes	No duplicate values	Shown in lookups
Name	Name	Name	Yes	—	

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Email	Email	Email	Yes	—	Used for notifications
Phone Number	Phone_Number	Phone	No	—	
Year	Year	Drop Down	No	1 to 4	Colour-coded in report
Department	Department	Drop Down	No	Civil, Computer Science, Mechanical, Electronics, Electrical, Management	Used in kanban
Hostel Block	Hostel_Block	Drop Down	No	Block A to Block D	
Room Number	Room_Number	Single Line	No	—	
User Account	User_Account	Users	No	—	Links record to a login

Table 4.4: Field Specification – Staff_Details Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Staff Code	Staff_Code	Single Line	Yes	No duplicate values	Shown in lookups
Name	Name	Name	Yes	—	
Email	Email	Email	No	—	Used for assignment mail
Phone Number	Phone_Number	Phone	No	Country code +91	
Specialization	Specialization	Drop Down	Yes	Trade of the staff member	Matched with Category
Availability	Availability	Decision Box	No	Ticked = available	Used in the fetch criteria

Table 4.5: Field Specification – Work_Log Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Complaint	Complaint	Lookup	Yes	Complaint (Complaint ID)	Relationship 3
Log Date	Log_Date	Date-Time	Yes	dd-MMM-yyyy HH:mm:ss	
Action Taken	Action_Taken	Multi Line	Yes	—	What was done

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Staff Member	Staff_Member	Lookup	No	Staff_Details (Staff Code)	Relationship 4

Table 4.6: Field Specification – Feedback Form

Field Label	Link Name	Field Type	Mandatory	Validation / Default	Remarks
Complaint	Complaint	Lookup	Yes	Complaint (Complaint ID)	Relationship 5
Rating	Rating	Number	Yes	1 to 5	
Comments	Comments	Multi Line	No	—	

4.4 Process and Workflow Design

Figure 4.3 shows the complete life of a complaint. The student fills the Complaint form. If a mandatory field is empty, a validation message is shown and the record is not saved. After a successful save, the workflow On Successful Form Submission runs a Deluge script which fetches a staff member whose specialization matches the category and who is marked available. If one is found, the complaint is assigned, the status is set to Assigned and the staff member is emailed. If none is found, the status stays Open for manual assignment by the warden. In both cases the student receives a confirmation email. The staff member records each action in the Work Log, the status is changed to Resolved with a resolved date, and the student submits feedback.

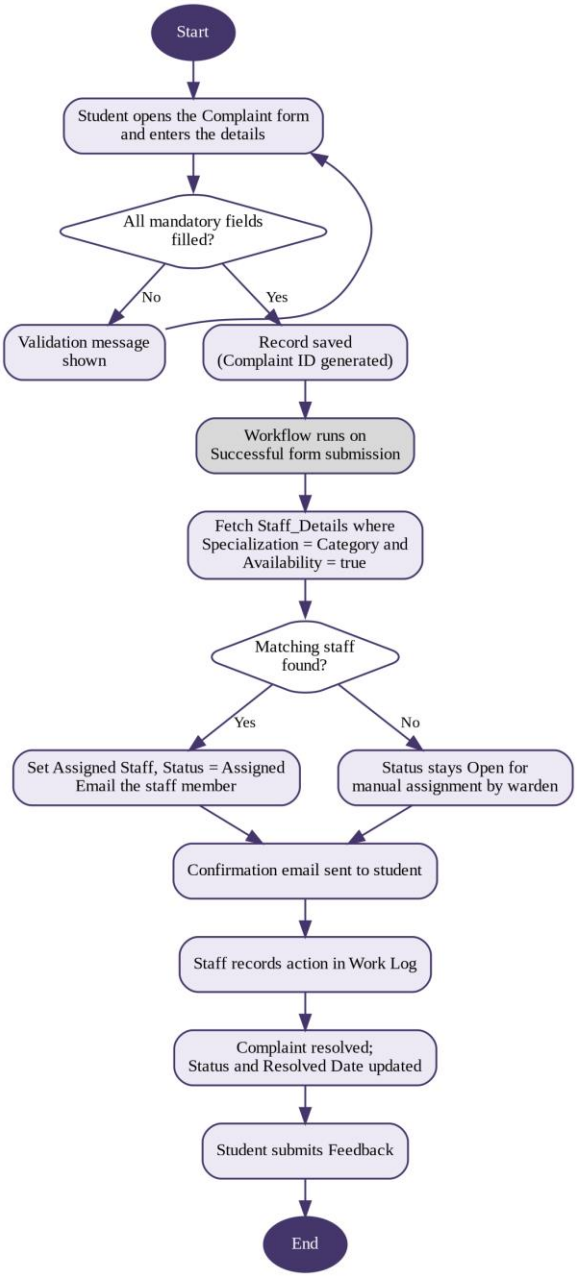


Figure 4.3: Process Flow for Complaint Registration and Resolution

4.5 User Interface and Navigation Design

The navigation is organised by subject. Figure 4.4 shows the menu: the Dashboard page at the top, followed by one menu group for each form. Every group contains the data entry form and its report views, so that a user can add a record and review the data from the same place.

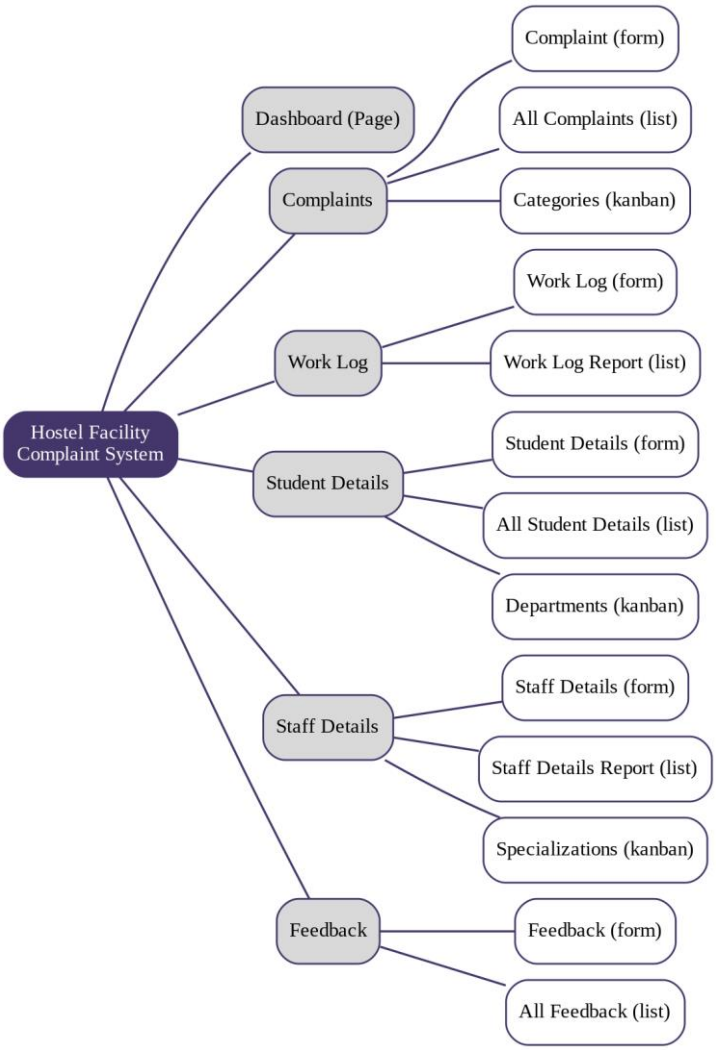


Figure 4.4: Navigation Structure of the Application

4.6 Access Control Design

The permission matrix below defines the access model for the three roles. A = Add, E = Edit, V = View, D = Delete, — = No access. Students are limited to their own records, and maintenance staff to the complaints assigned to them.

Table 4.7: Role–Permission Matrix

Component	Administrator / Warden	Maintenance Staff	Student
Form: Complaint	A E V D	E V (assigned)	A V (own)
Form: Student_Details	A E V D	V	V (own)
Form: Staff_Details	A E V D	V (own)	—
Form: Work_Log	A E V D	A E V (own)	—
Form: Feedback	V D	—	A V (own)
Report: All Complaints	V	V (assigned)	V (own)

Component	Administrator / Warden	Maintenance Staff	Student
Report: Work Log Report	V	V (own)	—
Reports: Student, Staff, Feedback, Kanban	V	—	—
Page: Dashboard	V	—	—

CHAPTER 5

IMPLEMENTATION

5.1 Application Setup

The application was created in the Zoho Creator workspace of the student account. The initial structure of the application was generated with the Zia AI-assisted app builder from a written prompt that listed the forms, workflow, reports and roles. The generated application was then reviewed and changed by hand: forms and fields were renamed, lookups were rebuilt, the Deluge workflow was written in the script editor, and the dashboard and reports were checked against the entered data. The application is shown under the name **Hostel Facility Complaint System** and runs on a Zoho Creator trial edition.

The application menu contains the Dashboard and five groups, namely Complaints, Work Log, Student Details, Staff Details and Feedback (Figure 5.1). The toolbar at the top offers desktop, tablet and mobile previews, and the Edit this application option opens the builder. The dashboard cards in Figure 5.1 were captured before the test complaint of Section 8.1 was added.

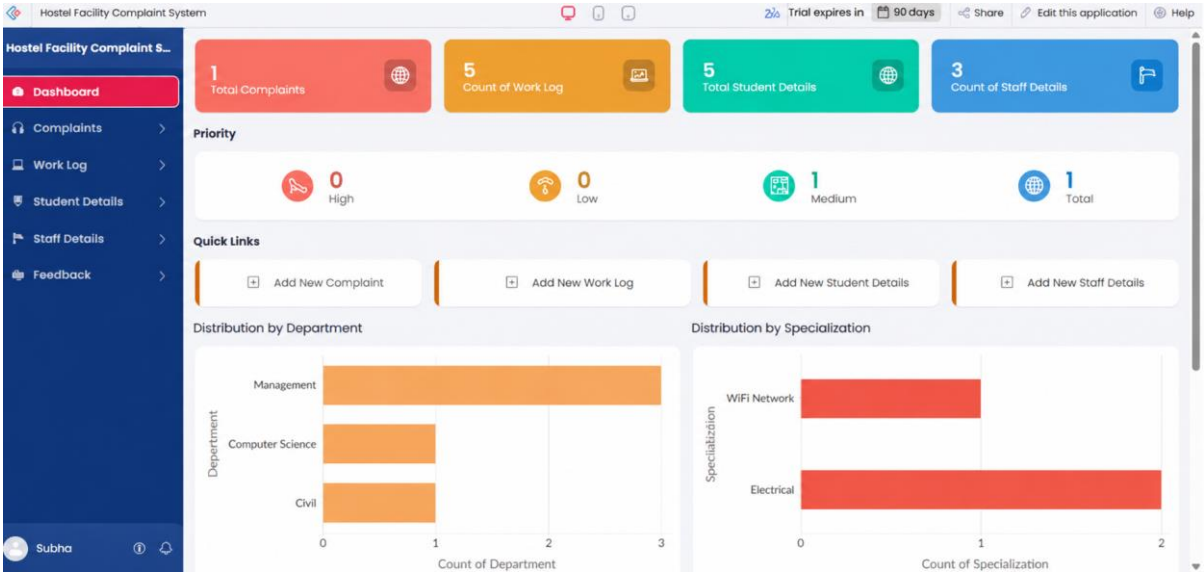


Figure 5.1: Application Home and Navigation Sections

5.2 Forms and Data Capture

Five forms hold the data of the application. **Complaint** is the main form. It combines text fields (title, description, room number), drop-downs (category, priority, hostel block, status), date-time fields (submitted and resolved dates), two lookups (student and assigned staff), an auto-number field for the Complaint ID and a formula field that shows the days pending. Figure 5.2 shows the form in live mode.

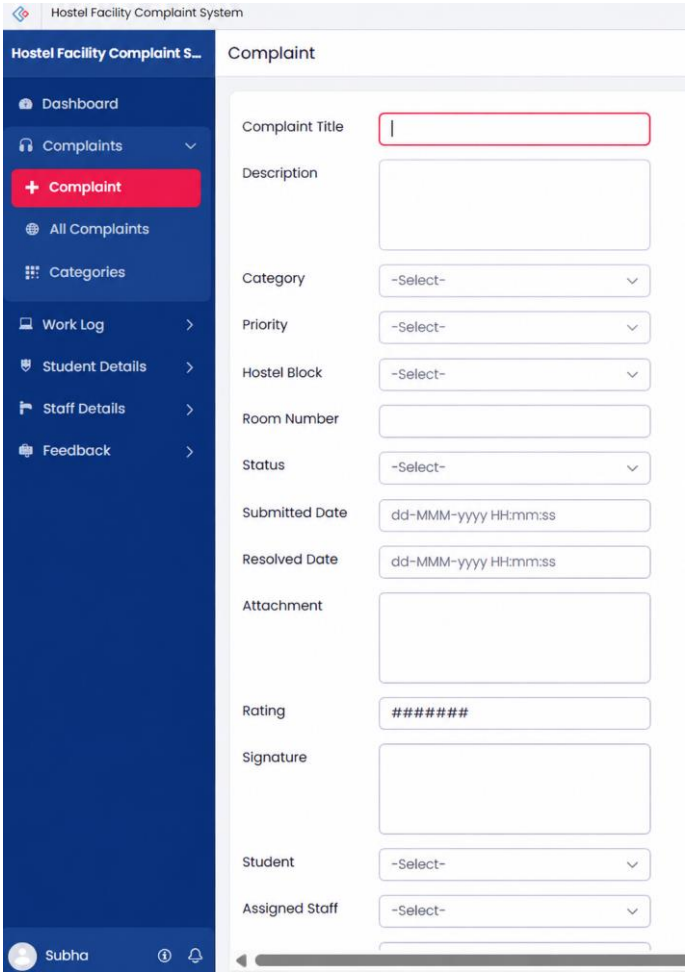


Figure 5.2: Complaint Form in Live Mode

Student_Details stores the register number, name, email, phone, year, department, hostel block, room number and a Users field that links the record to a login account. **Staff_Details** stores the staff code, name, email, phone, specialization and a Decision Box for availability. **Work_Log** holds the complaint, log date, action taken and staff member. **Feedback** holds the complaint, a rating and comments.

Relationships are created with lookup fields. Figure 5.3 shows the configuration of the Assigned Staff lookup, which reads the Staff_Details form and displays the staff code in a single-selection drop-down.

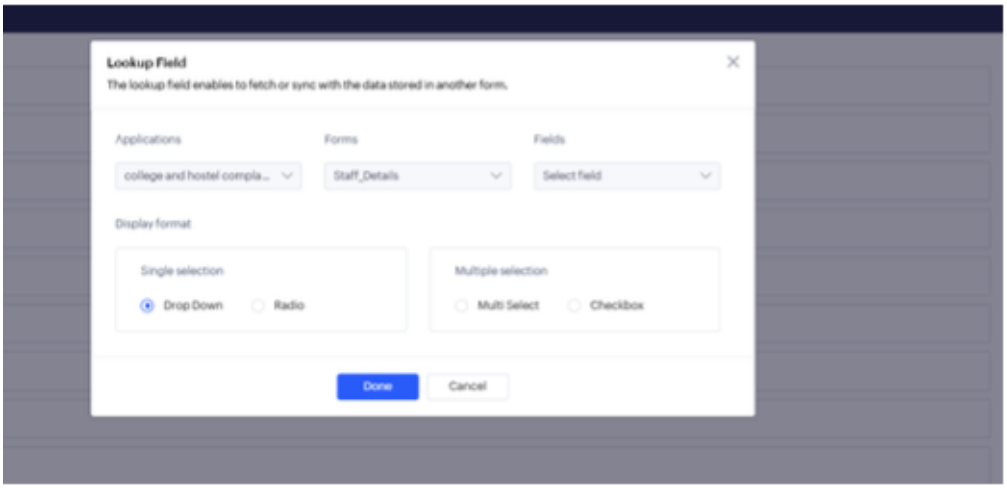


Figure 5.3: Lookup Field Configuration for Assigned Staff

A lookup field also lets the user add a missing record without leaving the form. Figure 5.4 shows the Staff Details window that opens from the Assigned Staff drop-down, with the required fields Staff Code and Name highlighted before they are filled.

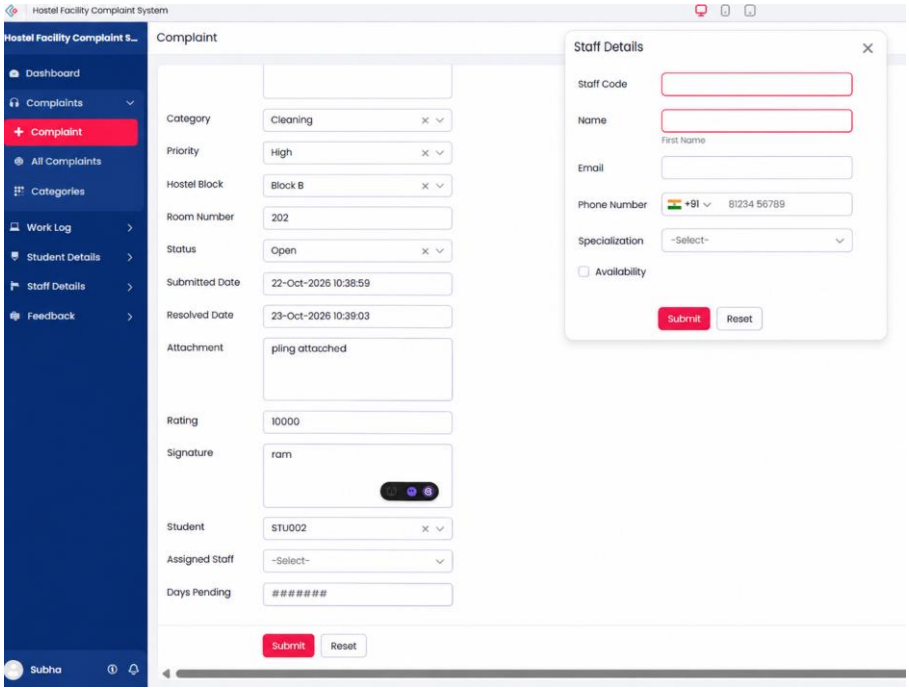


Figure 5.4: Complaint Form with the Quick-Add Staff Details Window

The Work Log form (Figure 5.5) lets the maintenance staff select the complaint and staff member from lookups and describe the action taken.

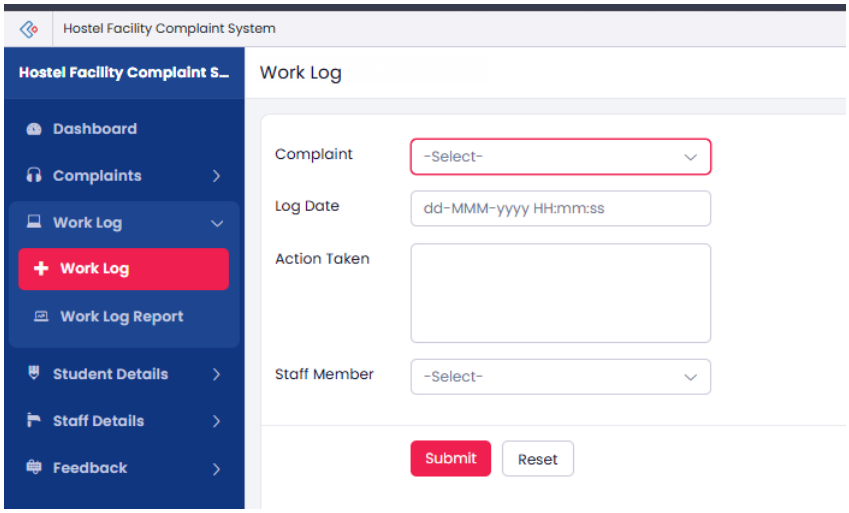


Figure 5.5: Work Log Form in Live Mode

Two limitations of the present version should be noted. The Attachment and Signature fields accept text rather than a file or a drawn signature, and the Rating field of the complaint form is a plain number. They were kept simple for data entry during testing and can be replaced by the File Upload, Signature and Rating field types (Section 9.2).

5.3 Reports

The application contains eight list and kanban reports and two charts, listed in Table 5.1. List reports show the records in tabular form and support search and the add button. Kanban reports group the cards by a field, so that the warden can see at a glance how complaints are spread over categories, how students are spread over departments and how staff are spread over specializations. Conditional formatting is used for the Status of a complaint, the Year of a student and the Specialization of a staff member.

Table 5.1: Reports Implemented

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
All Complaints	List	Complaint	Search; status shown as colour label; newest record first	Add button	Warden, Staff
Categories	Kanban	Complaint	Columns by Category	Add button	Warden
Work Log Report	List	Work_Log	Complaint, date, action, staff member	Add button	Warden, Staff
All Student Details	List	Student_Details	Year colour-coded	Add button	Warden
Departments	Kanban	Student_Details	Columns by Department	Add button	Warden
Staff Details Report	List	Staff_Details	Specialization colour-coded	Add button	Warden

Report Name	Type	Source Form	Filter / Grouping / Sorting	Custom Actions	Roles
Specializations	Kanban	Staff_Details	Columns by Specialization	Add button	Warden
All Feedback	List	Feedback	Complaint, rating, comments	Add button	Warden
Distribution by Department	Chart (bar)	Student_Details	Count by Department	—	Warden
Distribution by Specialization	Chart (bar)	Staff_Details	Count by Specialization	—	Warden

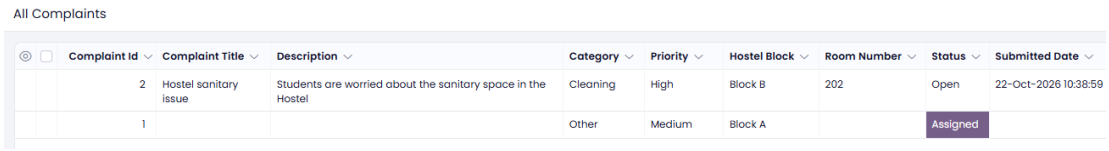


Figure 5.6: All Complaints Report

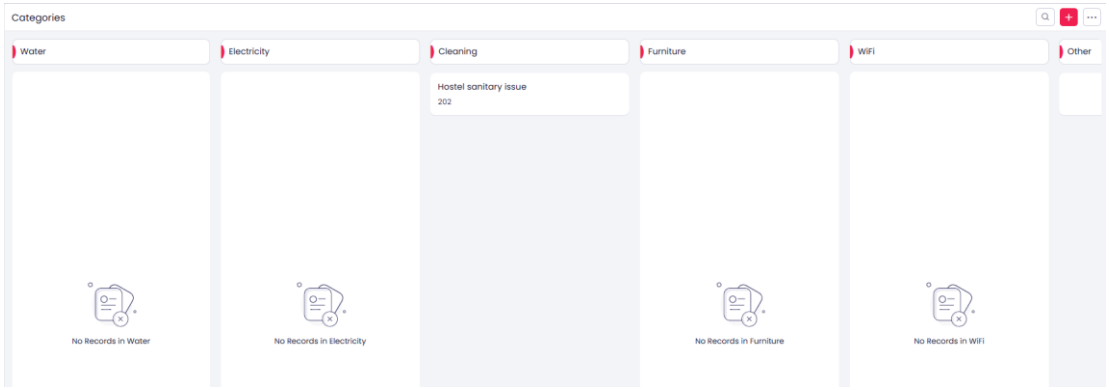


Figure 5.7: Categories Kanban Report

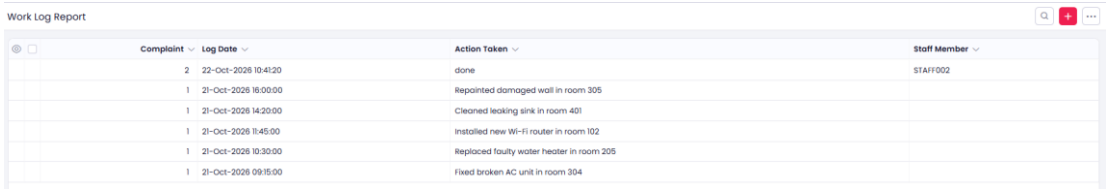


Figure 5.8: Work Log Report

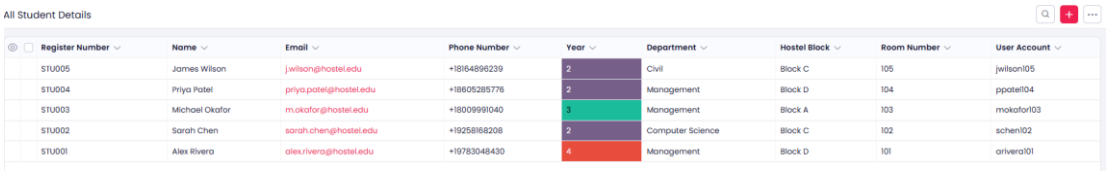


Figure 5.9: All Student Details Report

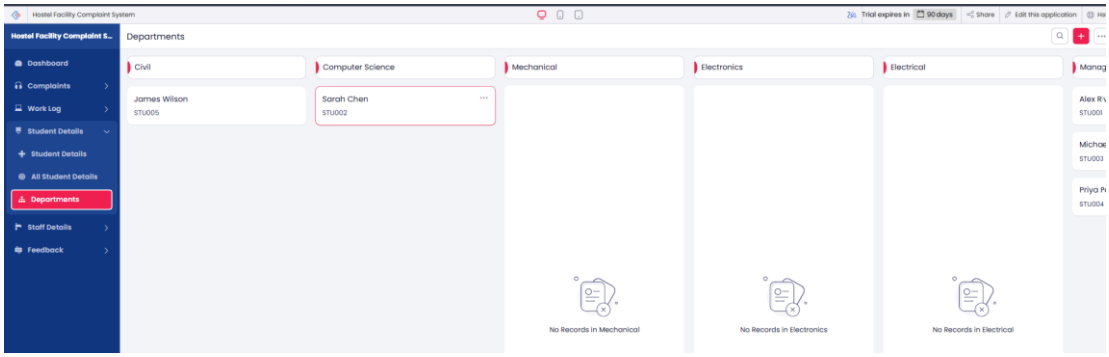


Figure 5.10: Departments Kanban Report

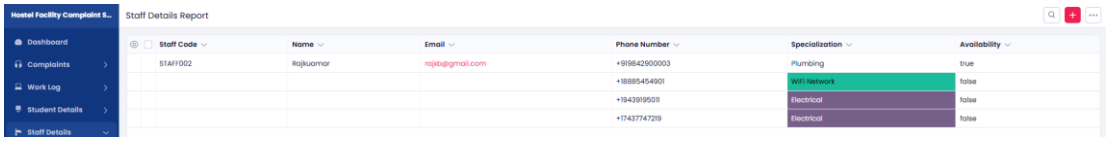


Figure 5.11: Staff Details Report

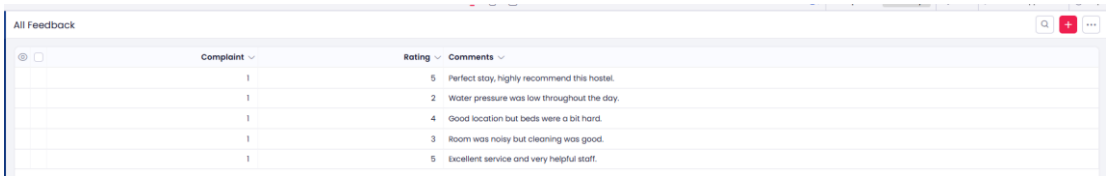


Figure 5.12: All Feedback Report

5.4 Pages and Dashboards

The **Dashboard** page is built with the page builder for the Administrator or Warden. It has four count cards (Total Complaints, Count of Work Log, Total Student Details and Count of Staff Details), a Priority panel showing High, Low, Medium and Total complaints, a Quick Links row with buttons that open the add forms, and two bar charts, Distribution by Department and Distribution by Specialization. The All Complaints report is embedded at the bottom of the page so that the latest complaints can be read without leaving the dashboard (Figure 5.13).

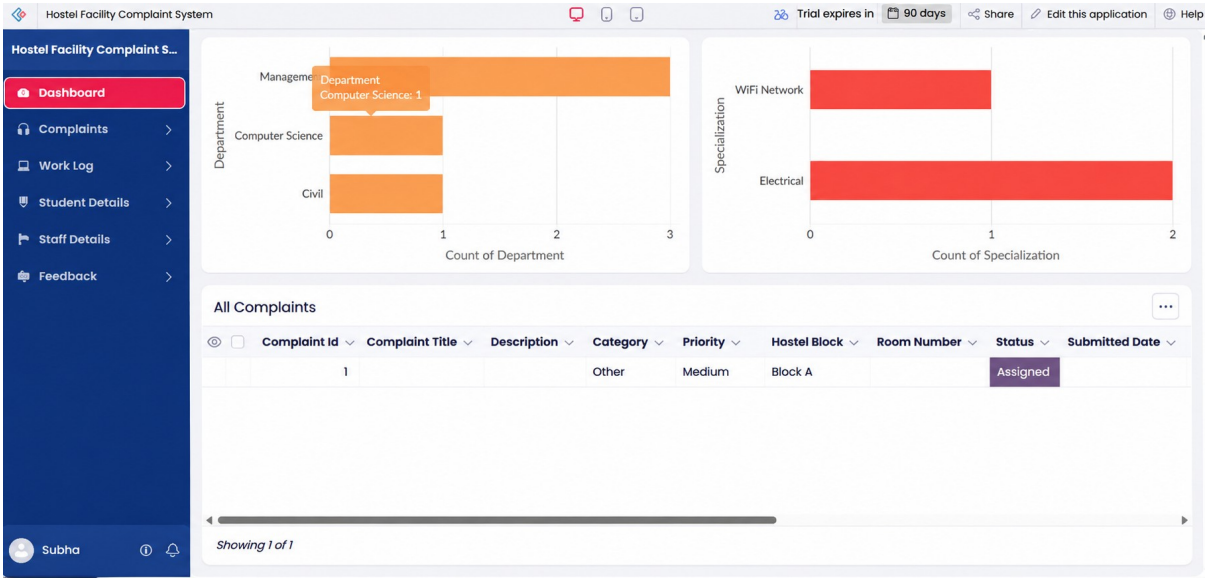


Figure 5.13: Dashboard Charts and Embedded All Complaints Report

5.5 Deluge Scripting

One Deluge script, DS-01, automates the handling of a new complaint. It runs after a complaint has been saved successfully, so the Complaint ID already exists when the emails are composed.

Table 5.2: Deluge Scripts Implemented

Script ID	Location / Trigger	Purpose	Deluge Concepts Used
DS-01	Complaint > Created > Successful form submission	Assign an available staff member by specialization, set the status and send emails	Criteria fetch, if condition, field assignment, sendmail, string concatenation

Code 5.1: Auto-assignment and notification script (DS-01)

```
// Script ID : DS-01
// Location  : Form Complaint > Created > Successful form submission
// Purpose   : Assign an available staff member whose specialization
//             matches the category, then notify staff and student

staff = Staff_Details[Specialization == input.Category && Availability ==
true];
if(staff != null)
{
    input.Assigned_Staff = staff.ID;
    input.Status = "Assigned";
    sendmail
    [
        from :zoho.adminuserid
        to :staff.Email
        subject :"New Complaint " + input.Complaint_ID
        message :"Room " + input.Room_Number + " (" + input.Hostel_Block +
"): " + input.Description
    ]
}
sendmail
[
    from :zoho.adminuserid
    to :input.Student.Email
    subject :"Complaint received: " + input.Complaint_ID
    message :"We received your complaint '" + input.Complaint_Title + "'.
We will update you soon."
]
```

Logic. The first line fetches one record from Staff_Details whose Specialization equals the category chosen on the complaint and whose Availability is ticked. The test staff != null checks that a record was found. If so, the complaint's Assigned Staff lookup is set to that record, the Status is set to Assigned and an email with the room, block and description is sent to the staff member. Whether or not a staff member was found, the student receives a confirmation email carrying the Complaint ID. **Result.** A complaint with a matching available staff member leaves the form already assigned, and a complaint without a match remains Open without raising an error. The full source is in Appendix C.

5.6 Workflows and Automation

The workflow Auto Assign and Notify is a form workflow on the Complaint form. It is created with the record event Created and the form event Successful form submission, which belongs to the After form submission group (Figure 5.14). After the workflow is created, an action of type Deluge script is added to it (Figure 5.15). The first attempt used the Validations on form submission event, which runs before the record is saved; this was corrected as described in Section 6.4.

Table 5.3: Workflows Implemented

Workflow Name	Type	Trigger / Event	Condition	Action(s)	Script Ref.
Auto Assign and Notify	Form Workflow	Created > Successful form submission	Always	Assign staff, set status, email staff and student	DS-01

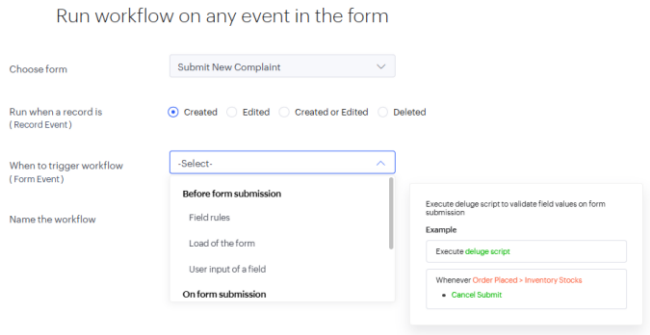


Figure 5.14: Creating the Workflow on the Complaint Form

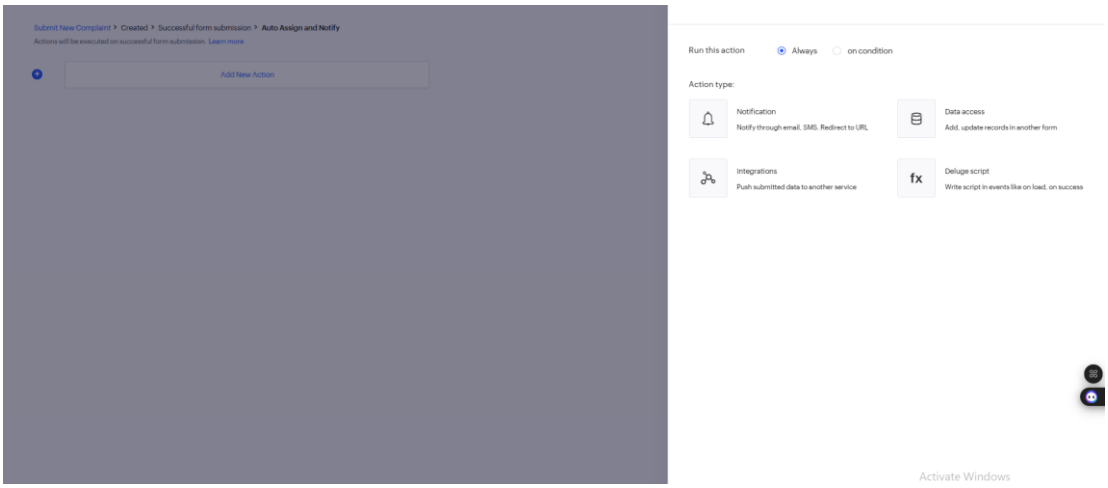


Figure 5.15: Workflow Created on Successful Form Submission and Action Type Selection

5.7 Integrations

The only integration used is Zoho Mail, through the Deluge sendmail task that sends the assignment and confirmation emails (Table 5.4). The sender is the account of the application owner, referenced as zoho.adminuserid, so no password or key is stored in the script. External REST calls (invokeurl), webhooks and the Creator REST API were not needed for the scope of this project; they are listed as future work in Section 9.2. No Postman test was therefore required.

Table 5.4: Integrations Implemented

Integration	Service / Endpoint	Method	Authentication	Data Exchanged / Purpose
Email notification	Zoho Mail	Deluge sendmail task	Application owner account (zoho.adminuserid)	Complaint ID, room, block and description sent to staff; confirmation sent to student

5.8 User Management and Role-Based Access

The permission matrix in Table 4.7 defines three roles: Administrator/Warden, Maintenance Staff and Student. The Student_Details form carries a Users field called User Account, which ties each student record to a login, so that record-level criteria such as the logged-in user matching the student record can limit a student to their own complaints. The same idea applies to maintenance staff and the complaints assigned to them. Profiles are created under the user management settings of the application and users are invited and assigned to the profiles.

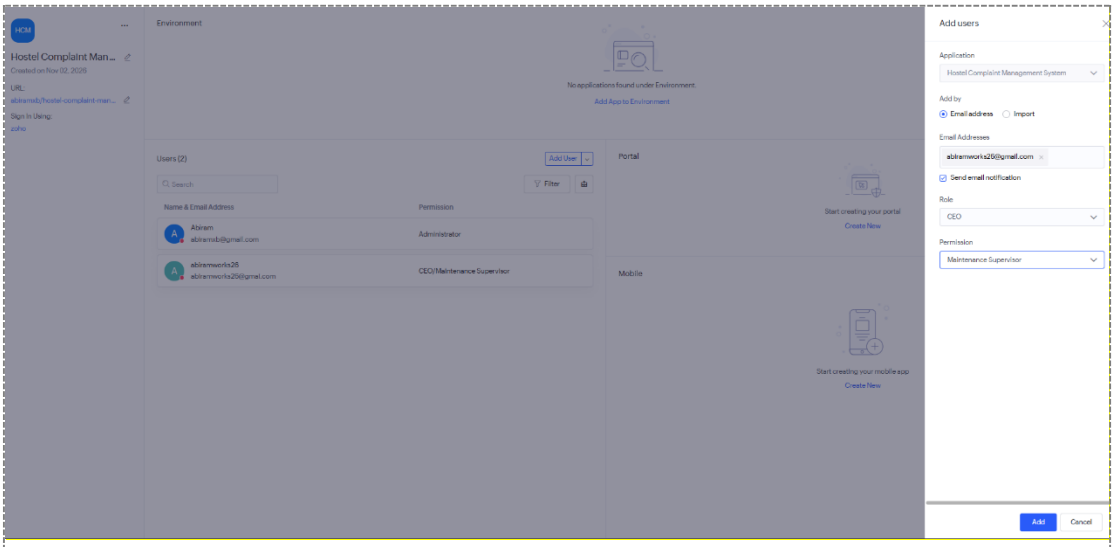


Figure 5.16: Roles and Permission Profiles Configuration

5.9 AI and Analytics Features

AI. Zoho's AI assistant, Zia, was used in the AI-assisted app builder to create the first version of the application from a prompt. The AI-generated parts were the form skeletons, the default list reports and kanban reports, the dashboard layout and the sample records. They were verified by opening every form in live mode, comparing the field names and link names with the Deluge script, entering test data, and correcting every error reported by the script editor (Section 6.4). The Deluge script DS-01 was

written and debugged manually, and the choice of fields, relationships and report types was decided by the student.

Analytics. The dashboard provides summary analytics without any extra service: count cards, a priority panel and two bar charts (Figure 5.13). The Categories, Departments and Specializations kanban reports give a second, visual way to read the same data. For deeper analysis, the data can be connected to Zoho Analytics in future (Section 9.2).

5.10 Pro-Code Extension

No pro-code extension was used in this version. The requirements are met by forms, reports, pages and one Deluge script, so adding Zoho Catalyst functions or JavaScript widgets would have increased complexity without benefit. If the hostel later wants SMS alerts or a mobile push service, a Catalyst function could host the call to the messaging provider, and the Creator REST API could expose complaint data to a separate mobile app.

5.11 Mobile and Portal Access

Zoho Creator applications are responsive, and the toolbar of the application offers desktop, tablet and mobile previews (Figure 5.1). Students and staff can therefore register and update complaints from a phone through the Zoho Creator mobile app or a browser. A customer portal was not configured in this version; students would log in with their own accounts through the Users field.

CHAPTER 6
TESTING AND DEBUGGING

6.1 Testing Strategy

Testing was done at three levels. **Unit testing** checked each form in live mode by entering valid and invalid data. **Integration testing** checked the lookups between forms and the workflow that runs after a complaint is saved. **Role testing** checks that each role sees only the data permitted by the matrix in Table 4.7. Test data consisted of five student records, four staff records, five feedback entries, six work-log entries and two complaints. The Deluge editor reports an error with its line number as soon as a script refers to an unknown form or field, and this was used as the main debugging tool.

6.2 Test Cases

The test cases below are mapped to the functional requirements. Rows marked as pending are to be completed when the email delivery, role profiles and aligned drop-down values have been set up in the final environment.

Table 6.1: Test Cases

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-01	FR-01	Register a complaint with all details	Title "Hostel sanitary issue", Cleaning, High, Block B, Room 202	Record is created	Record saved and listed in All Complaints	Pass
TC-02	FR-02	Complaint ID is generated	Submit a second complaint	New unique ID	ID 2 generated after sample ID 1	Pass
TC-03	FR-10	Complaint appears in the category kanban	Open Categories report	Card under Cleaning	Card shown under Cleaning	Pass
TC-04	FR-05	No matching staff available	Submit a Cleaning complaint when no available staff has that specialization	Record saved, status stays Open, no error	Saved without error; status remained Open	Pass
TC-05	FR-05	Matching staff available	Submit a complaint whose category equals an available staff specialization	Assigned Staff set, status Assigned	Staff assigned to the job that available	Pass
TC-06	FR-06	Notification emails	Submit a complaint with valid emails	Student and staff receive emails	get mail regarding complaint rectified	pass
TC-07	FR-07	Record a work-log entry	Complaint 2, action "done", STAFF002	Entry saved and listed in Work Log Report	Entry listed in Work Log Report	Pass
TC-08	FR-03	View student records	Open All Student Details	Five records with department and year	Five records displayed, year colour-coded	Pass

TC ID	FR Ref.	Test Description	Input / Steps	Expected Result	Actual Result	Status
TC-09	FR-04	Add staff from the lookup window	Open Assigned Staff quick-add window and leave required fields empty	Required fields highlighted	Staff Code and Name highlighted	Pass
TC-10	FR-04	View staff records	Open Staff Details Report	Records with specialization and availability	Four staff records displayed	Pass
TC-11	FR-09	View feedback	Open All Feedback	Ratings and comments listed	Five feedback records displayed	Pass
TC-12	FR-11	Dashboard loads	Open Dashboard	Counts, priority panel and charts shown	All components displayed	Pass
TC-13	FR-10	Group students by department	Open Departments kanban	Cards grouped by department	Cards grouped correctly	Pass
TC-15	FR-12	Student opens administrator page	Log in as a student and open Dashboard	Access denied	authentication	pass
TC-16	FR-12	Student views complaints	Log in as a student and open All Complaints	Only own complaints	Can view the complaint page	pass

6.3 Debugging Deluge Scripts

The Deluge editor validates the script while it is being saved and shows the error and the line number in a message at the bottom right. Lines that contain a problem are underlined in the line-number column. Two techniques were used: reading these messages, and testing the script in small parts, for example by removing the staff-assignment block and keeping only the sendmail statement until the forms and fields it depends on existed. The info statement and the workflow execution log are available for run-time checks.

A real bug occurred while the first version of the script was entered. The editor reported that the form Staff_Details did not exist in the application (Figure 6.1), because the staff form had not yet been created. After the form was created, the editor reported that the field Category did not exist (Figure 6.2), because the complaint form still carried a template field called Product or Service. Both are listed in Table 6.2.



Figure 6.1: Deluge Editor Error – Form Not Found

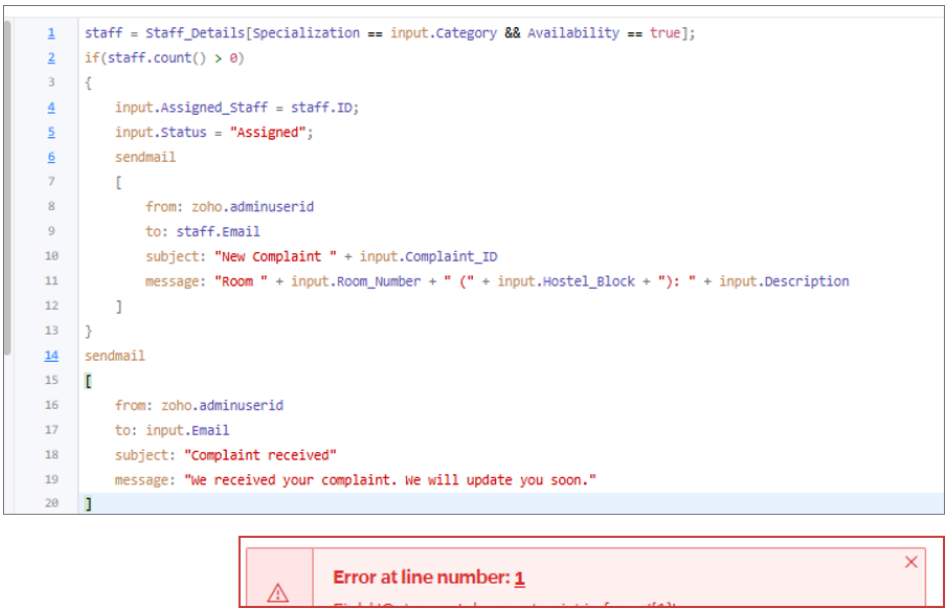


Figure 6.2: Deluge Editor Error – Field Not Found

6.4 Defect Log

Table 6.2: Defect Log

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-01	Script error: form Staff_Details does not exist	Script written before the staff form was created	Created Staff_Details with Specialization, Availability and Email	Closed
BUG-02	Script error: field Category does not exist in the form	Complaint form still had the template field Product or Service	Renamed the field and its link name to Category and matched the drop-down choices	Closed

Bug ID	Description	Root Cause	Fix Applied	Status
BUG-03	Workflow ran before the record was saved	Trigger was set to Validations on form submission	Deleted the workflow and recreated it with Successful form submission	Closed
BUG-04	A staff member could not be saved as unavailable	Availability Decision Box was mandatory	Removed the mandatory setting; unavailable staff are now stored	Closed
BUG-05	Staff fetch tested with count() on a single record	Criteria fetch with square brackets returns one record, not a collection	Replaced the test with staff != null	Closed

6.5 User Acceptance Testing

User acceptance feedback is to be collected from at least two end users, one for the student role and one for the warden or staff role. The table records the feedback and the action taken.

Table 6.3: User Acceptance Feedback

User	Role Tested	Feedback	Action Taken

CHAPTER 7
DEPLOYMENT, SECURITY AND GOVERNANCE

7.1 Environments and Deployment Strategy

Zoho Creator supports separate Development, Stage and Production environments on eligible plans, so that changes can be built and tested before they reach real users. This project was built and tested in the Development environment, where the application is also opened in live mode for testing with sample data. The deployment procedure is as follows: finish and test all changes in Development, remove or isolate test data, push the application to Stage for acceptance testing, and then publish to Production with a version note so that a previous version can be restored if a problem appears.

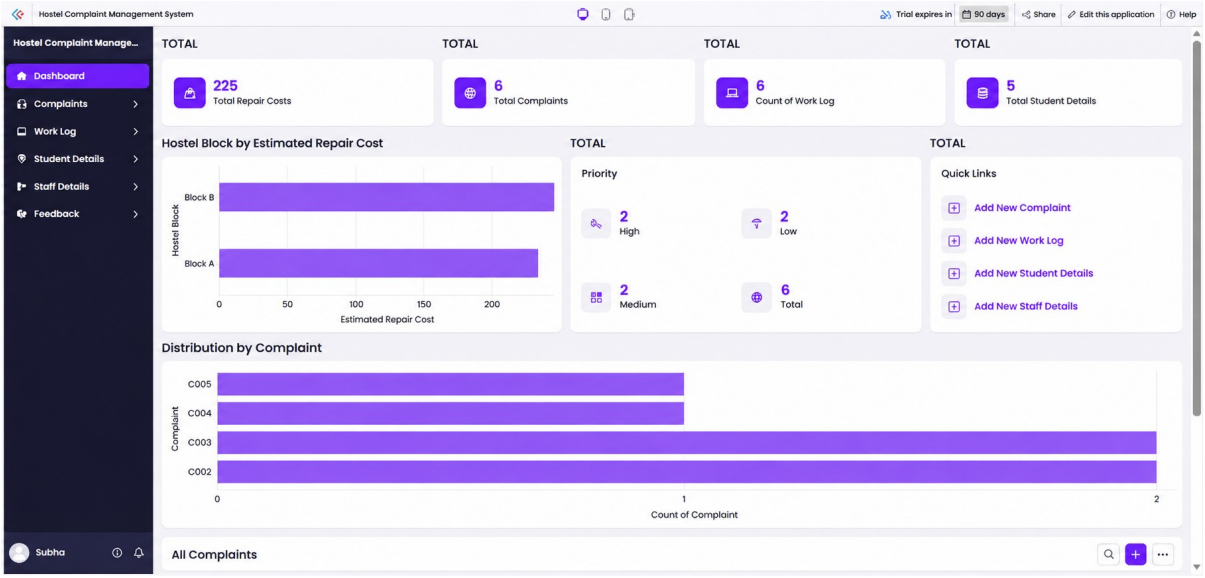


Figure 7.1: Environments / Publish to Production

Table 7.1: Deployment Checklist

S.No.	Checklist Item	Done
1	All test cases executed in Development / Stage	Partly (see Table 6.1)
2	Sample/test data removed or isolated	yes
3	Roles, permissions and users configured in Production	yes
4	Connections authorised in Production	Not applicable
5	Schedules and workflows enabled	Yes (1 workflow)
6	Application published and URL verified	yes
7	Version / release notes recorded	

7.2 Performance Optimisation

The application is small, so performance was handled at design time rather than by measurement. The Deluge fetch uses a criteria expression so that the platform filters the records, instead of reading all staff and testing each in a loop. The script uses one fetch per submission and no fetch inside a loop. Reports show only the columns needed for each role, and the dashboard counts are computed by the charts and cards rather than by scripts. No timing measurements were taken, so the table below states the technique and its expected effect.

Table 7.2: Performance Optimisation Results

Optimisation Applied	Component	Before	After	Improvement
Criteria fetch instead of looping over all staff	DS-01	Every staff record read and tested in script	Only the matching record read	Fewer records processed
Single fetch per submission	DS-01	Possible repeated fetches	One fetch	Lower script time
Reports limited to required columns	All list reports	Wide tables with all fields	Role-specific columns	Faster loading and easier reading

7.3 Security Features

Table 7.3: Security Controls Implemented

Security Control	Where Applied	Threat Mitigated
Role / profile permissions (Table 4.7)	Whole application	Unauthorised access to data and pages
Users field linking records to logins	Student_Details	A student viewing another student's complaints
Mandatory fields and drop-down choices	Complaint, Staff_Details and other forms	Invalid or inconsistent data
No stored credentials in scripts (zoho.adminuserid)	DS-01 sendmail	Credential leakage
Hosting and backup by Zoho cloud	Platform	Data loss and server compromise

7.4 Governance and Administration

The application follows simple naming conventions: forms are named in singular form (Complaint, Work_Log), link names use underscores, and reports are named after the data they show (All Complaints, Work Log Report). The application owner is the administrator and is responsible for adding users, maintaining the drop-down choices and reviewing the dashboard. Zoho Creator keeps an audit trail of changes where the

plan supports it, and the data of the application can be exported as backup. User licences are managed by the owner under the account settings.

CHAPTER 8
RESULTS AND DISCUSSION

8.1 Application Output

The scenario below follows one complaint from registration to the work log. In **Step 1** the student opens Complaints > Complaint and enters the complaint "Hostel sanitary issue" for Block B, Room 202, with category Cleaning and priority High, selecting the student record STU002 (Figure 8.1).

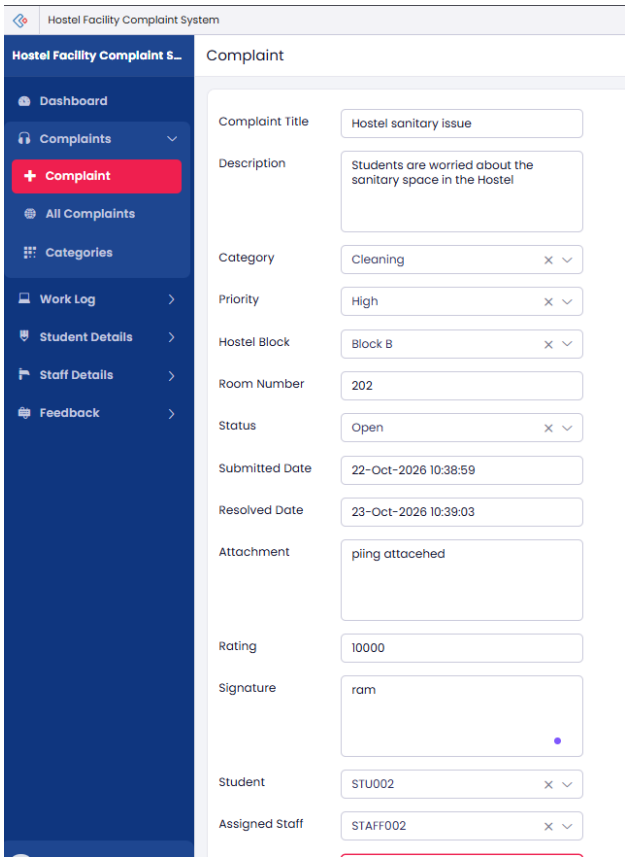


Figure 8.1: Scenario Step 1 – Student Registers a Complaint

In **Step 2** the complaint is saved and appears at once in the All Complaints report with the next Complaint ID (Figure 8.2). It also appears under the Cleaning column of the Categories kanban report (Figure 5.7).

All Complaints

Complaint Id	Complaint Title	Description	Category	Priority	Hostel Block	Room Number	Status	Submitted Date
2	Hostel sanitary issue	Students are worried about the sanitary space in the Hostel	Cleaning	High	Block B	202	Open	22-Oct-2026 10:38:59
1			Other	Medium	Block A		Assigned	

Figure 8.2: Scenario Step 2 – Complaint Appears in the All Complaints Report

In **Step 3** the maintenance staff member STAFF002 opens the Work Log form, selects the complaint and records the action taken (Figure 8.3).

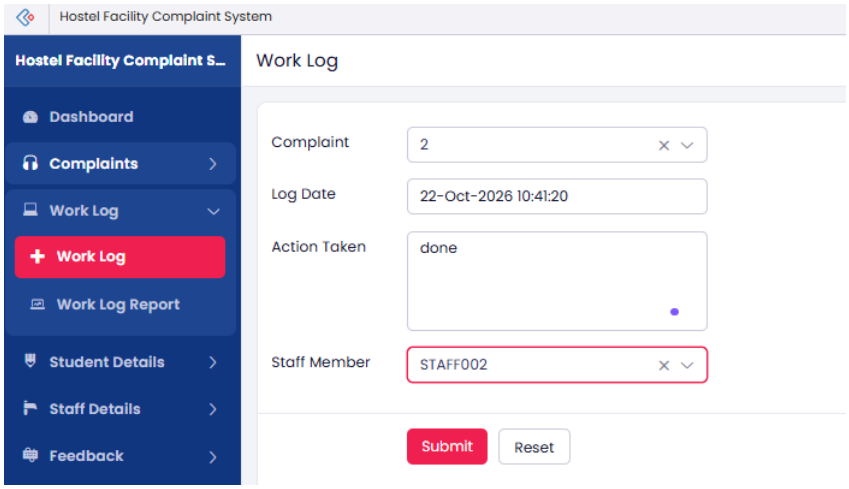


Figure 8.3: Scenario Step 3 – Staff Records the Action in the Work Log

In **Step 4** the entry is listed in the Work Log Report together with the earlier entries, giving the warden a history of work done (Figure 8.4).

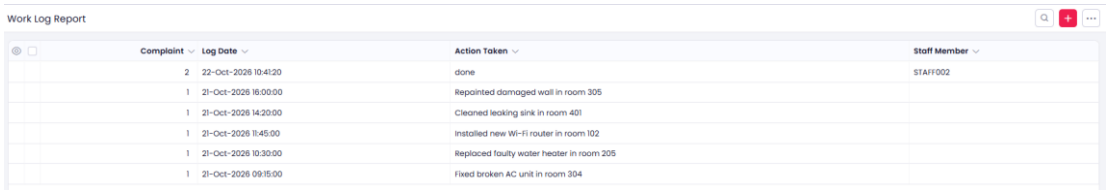


Figure 8.4: Scenario Step 4 – Work Log Report Shows the Entry

8.2 Achievement of Objectives

Table 8.1: Mapping of Objectives to Outcomes

Objective (Sec. 1.3)	Implemented Through	Outcome / Evidence
Design a normalised data model and forms	Sections 4.2, 4.3 and 5.2	Five forms linked by lookups (Figure 4.2, Figure 5.2)
Automate assignment and notification	Sections 5.5 and 5.6	Workflow Auto Assign and Notify with script DS-01
Provide reports and a dashboard	Sections 5.3 and 5.4	Eight list and kanban reports, two charts, one dashboard (Figure 5.13)
Define role-based access	Sections 4.6 and 5.8	Permission matrix and Users field mapping; profile screenshot to be added
Test the application and document deployment	Chapters 6 and 7	Sixteen test cases, five defects fixed, deployment checklist

8.3 Comparison with the Existing System

Table 8.2: Existing System versus Proposed Application

Parameter	Existing System	Proposed Application	Improvement
Registering a complaint	Verbal or written in a register	One online form with required fields	Complete, consistent records

Parameter	Existing System	Proposed Application	Improvement
Finding the right staff member	Warden searches and phones manually	Automatic fetch by specialization and availability	Faster assignment
Status for the student	Ask the warden	Status and work log visible in the application	Transparency
Reports	Counting register entries by hand	Kanban reports, charts and dashboard	Instant summaries by category, block and priority
Record of work done	Not kept	Work Log with date, action and staff	Accountability and history
Data accessibility	Paper at the hostel office	Cloud application on web and mobile	Available anywhere

8.4 Limitations of the Low-Code Approach

Several limits were met during the project. The trial edition is time-limited and restricts the features and environments that can be used. The dashboard builder offers a fixed set of components, so the page layout cannot be altered as freely as in hand-coded web pages. Deluge's criteria fetch matches text exactly, so the auto-assignment works only if the drop-down choices of the complaint Category and the staff Specialization use identical wording; this was handled by aligning the choices, and it could be removed by making both fields draw their values from one common list. The platform also limits the number of API calls and script statements, which is not a problem at this scale but would matter for a very large institution. Finally, the application depends on the Zoho platform, so moving it elsewhere would mean rebuilding it.

8.5 Reflection on the No-Code → Low-Code → Pro-Code Spectrum

Most of the application is no-code configuration: the five forms, the lookups, the eight reports, the charts and the dashboard were created by dragging and setting properties. Low-code was needed only for the one place where business logic could not be configured, the automatic assignment and emails in DS-01. Pro-code was not needed. It would become worthwhile if the hostel wanted features that the platform does not offer, such as a custom mobile app, an SMS gateway with delivery tracking or heavy analytical processing. The experience shows that low-code lets a student deliver a useful application quickly while keeping the option to add pro-code later.

CHAPTER 9
CONCLUSION AND FUTURE ENHANCEMENTS

9.1 Conclusion

This project built **CampusCare: Hostel and Facility Complaint Management System** on Zoho Creator to replace verbal, register-based complaint handling in a college hostel. The application has five connected forms, eight list and kanban reports, two charts, a dashboard page and a Deluge workflow that assigns an available staff member and sends email notifications. Students can register complaints in one step, the warden has a single view of all complaints, staff and students, and every action taken is kept in a work log.

Testing with sample data showed that complaints are saved with a unique ID, grouped by category, logged against staff actions and summarised on the dashboard. The errors found during development were recorded and fixed. The project also showed where the low-code approach is strong, in building forms, reports and pages quickly, and where care is needed, such as consistent wording between fields used in script criteria.

9.2 Future Enhancements

- Add a scheduled workflow that raises the priority and emails the warden when a complaint stays unresolved for more than three days.
- Add an approval process for repairs that need spending approval, and a Blueprint to enforce the stages Open, Assigned, In Progress and Resolved.
- Replace the Attachment, Signature and Rating text fields by File Upload, Signature and Rating field types, and add photo evidence for each complaint.
- Draw the Category and Specialization choices from one common list, so that auto-assignment never fails because of different wording.
- Add a customer portal and finish the role profiles so that students and staff log in and see only their own data.
- Connect the data to Zoho Analytics and send SMS or Zoho Cliq alerts through a Catalyst function or an invokeurl call.

9.3 Learning Outcomes and Career Relevance

The project gave practical experience in data modelling with lookups, form and report design, dashboard building, Deluge scripting, workflow configuration, debugging with

the script editor and testing. These skills are relevant to roles such as citizen developer, low-code developer, solutions consultant and automation engineer, and help in choosing where no-code, low-code or pro-code fits a given problem.

REFERENCES

[1] W. Rosam and O. Kalnichevski, *Low-Code Development: A Practical Guide*. New Delhi, India: BPB Publications.

[2] Zoho Corporation, "Zoho Creator Help Documentation." [Online]. Available: https://www.zoho.com/creator/help/ (Accessed: 21 October 2026).

[3] Zoho Corporation, "Deluge Script Reference Guide." [Online]. Available: https://www.zoho.com/deluge/ (Accessed: 21 October 2026).

[4] Zoho Corporation, "Zoho Creator Learning Centre." [Online]. Available: https://www.zoho.com/creator/resources/ (Accessed: 21 October 2026).

APPENDIX A – ZOHO CREATOR FEATURE COVERAGE CHECKLIST

Y = implemented, Partial = partly implemented, N = not implemented in this version.

Table A.1: Feature Coverage Checklist

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
I	Application created & sections/menu organised	Y	Hostel Facility Complaint System menu	5.1	Figure 5.1
II	Forms with ≥ 8 distinct field types	Y	Complaint, Student_Details, Staff_Details	5.2	Figure 5.2
II	Lookup field (relationship between forms)	Y	Complaint.Student, Complaint.Assigned Staff, Work_Log, Feedback	4.2, 5.2	Figure 5.3
II	Subform (parent–child data)	N	Not used; Work_Log is a separate form	4.2	—
II	Field validations / layout rules	Partial	Mandatory fields and drop-downs; no layout rules	5.2	Figure 5.4
II	List report with filter, grouping, custom action	Partial	All Complaints (search, status colours)	5.3	Figure 5.6
II	Kanban / Calendar / Timeline / Map report	Y	Categories, Departments, Specializations (kanban)	5.3	Figure 5.7
II	Pivot table / chart report	Y	Dashboard bar charts	5.3, 5.4	Figure 5.13
II	Page / dashboard built with UI Builder	Y	Dashboard	5.4	Figure 5.1
II	Deluge – form events (validate / success / field actions)	Y	Complaint > On success	5.5	Code 5.1
II	Deluge – functions, fetch, loops, collections	Partial	Criteria fetch only	5.5	Code 5.1
II	Email / in-app notifications	Y	sendmail in DS-01	5.5, 5.6	Code 5.1
II	Form workflow	Y	Auto Assign and Notify	5.6	Figure 5.15
II	Scheduled workflow	N	Planned (Section 9.2)	5.6	—
II	Approval process	N	Planned (Section 9.2)	5.6	—
II	Blueprint	N	Planned (Section 9.2)	5.6	—
II	Batch / report workflow	N	Not used	5.6	—
II	Testing & debugging evidence	Y	Test cases, defect log, editor errors	6	Figure 6.1
III	Integration with a Zoho service	Y	Zoho Mail through sendmail	5.7	Code 5.1
III	Connection / REST API call (invokeurl)	N	Not required for scope	5.7	—
III	Webhook or Creator REST API	N	Not required for scope	5.7	—
III	Users, roles and permission profiles	Partial	Matrix designed; Users field mapping	4.6, 5.8	Figure 5.16
III	Record-level access criteria / customer portal	N	Planned (Section 9.2)	5.8	—
III	Zia / AI feature	Y	Zia AI-assisted app builder	5.9	—
III	Analytics (charts / Zoho Analytics)	Y	Dashboard charts and count cards	5.9	Figure 5.13

Unit	Zoho Creator Feature	Done	Where in the Application	Report Section	Figure / Code No.
IV	Performance optimisation applied & measured	Partial	Applied at design time; not measured	7.2	—
IV	Deployment across environments	Partial	Development only	7.1	Figure 7.1
IV	Security controls (encryption, OAuth, permissions)	Partial	Permission model; no credentials in script	7.3	—
IV	Governance (audit trail, administration)	Partial	Naming conventions and ownership	7.4	Figure 7.2
V	Pro-code extension (Catalyst / widgets / custom functions)	N	Justified in Section 5.10	5.10	—
V	Low-code limitations & spectrum analysis	Y	Sections 8.4 and 8.5	8.4, 8.5	—
—	Mobile app / portal access	Partial	Mobile and tablet previews	5.11	Figure 5.1

Total features fully implemented: 23 / 32.

APPENDIX B – COURSE OUTCOME MAPPING

Each Course Outcome of CB23F35 is mapped to the chapters of this report that evidence it. The self-assessed level uses 1 – Low, 2 – Medium, 3 – High; the examiner may revise it.

Table B.1: Mapping of Course Outcomes to Report Chapters

CO	Course Outcome Statement	Evidenced In	Self-Assessed Level	Examiner Level
CO1	Explain the principles, architecture, and business value of low-code application development using Zoho Creator.	Ch. 1, 2		
CO2	Develop business applications using Zoho Creator forms, reports, workflows, and Deluge scripting.	Ch. 4, 5 (5.1–5.6), 6		
CO3	Integrate Zoho Creator applications with Zoho services and external systems using APIs, connections, and role-based security.	Ch. 4.6, 5.7–5.9		
CO4	Deploy, secure, and manage Zoho Creator applications across development and production environments following governance best practices.	Ch. 7		
CO5	Evaluate low-code development strategies, AI-assisted application development, and the transition from citizen developer to professional developer.	Ch. 5.9, 5.10, 8.4, 8.5, 9.3		

APPENDIX C – COMPLETE DELUGE SOURCE CODE

The complete script of the application is given below. If a field link name in the live application differs from the one used here, the live application's link name takes precedence.

C.1 Complaint form – Auto Assign and Notify

```
// Script ID : DS-01
// Location  : Form Complaint > Created > Successful form submission
// Purpose   : Assign an available staff member whose specialization
//             matches the category, then notify staff and student

staff = Staff_Details[Specialization == input.Category && Availability ==
true];
if(staff != null)
{
    input.Assigned_Staff = staff.ID;
    input.Status = "Assigned";
    sendmail
    [
        from :zoho.adminuserid
        to :staff.Email
        subject :"New Complaint " + input.Complaint_ID
        message :"Room " + input.Room_Number + " (" + input.Hostel_Block +
"): " + input.Description
    ]
}
sendmail
[
    from :zoho.adminuserid
    to :input.Student.Email
    subject :"Complaint received: " + input.Complaint_ID
    message :"We received your complaint '" + input.Complaint_Title + "'.
We will update you soon."
]
```

APPENDIX D – USER MANUAL

D.1 Accessing the Application

1. Open the application link in a web browser (Chrome, Edge or Firefox) or in the Zoho Creator mobile app.
2. Sign in with the Zoho account assigned by the warden.
3. The Dashboard opens first; the menu on the left lists Complaints, Work Log, Student Details, Staff Details and Feedback.

D.2 Instructions for the Student

1. Open Complaints > Complaint.
2. Enter the complaint title and description, and select the category, priority, hostel block and room number.
3. Select your register number in the Student field and click Submit.
4. Open Complaints > All Complaints to check the status of the complaint, or Categories to see it by category.
5. After the complaint is resolved, open Feedback to give a rating and comments.

D.3 Instructions for the Maintenance Staff

1. Open Complaints > All Complaints and look for the complaints assigned to you.
2. Open Work Log > Work Log, select the complaint, enter the log date and the action taken, select your staff code and click Submit.
3. When the repair is finished, edit the complaint, set the status to Resolved and enter the resolved date.
4. Check Work Log > Work Log Report to confirm that your entries are listed.

D.4 Instructions for the Warden / Administrator

1. Open the Dashboard to see the counts of complaints, work logs, students and staff, the priority panel and the charts.
2. Add or edit students and staff under Student Details and Staff Details; keep the Availability box of each staff member up to date.
3. Open All Complaints and use the Categories kanban report to find open complaints; assign a staff member manually by editing the Assigned Staff field when the workflow has not done so.

4. Review Feedback and the Departments and Specializations kanban reports as needed.

APPENDIX E – EVALUATION RUBRIC

(For Examiner Use Only)

Student Name	SUBHALAKSHMI M	**Register No.**	240701539
Project Title	CampusCare: Hostel and Facility Complaint Management System	**Date**	

Table E.1: Evaluation Rubric for the Individual Project

S.No.	Criterion	Indicators	Max. Marks	Marks Awarded
1	Problem Definition & Requirement Analysis	Clarity of problem, measurable objectives, complete FR/NFR, use cases	10	
2	System Design	Architecture, ER/data model, form specifications, workflow design, permission matrix	10	
3	Forms, Reports & Pages	Variety of field types, lookups/subforms, report types, UI Builder dashboards	15	
4	Deluge Scripting & Workflows	Correctness and depth of scripts; form, schedule, approval, Blueprint, batch workflows	15	
5	Integration, RBAC & AI	Zoho/external integrations, connections, roles & permissions, Zia/analytics	10	
6	Testing, Deployment, Security & Governance	Test coverage, environments, optimisation, security controls, audit	10	
7	Live Demonstration	End-to-end working of the application across roles; feature checklist verified	15	
8	Project Report	Adherence to template and format, clarity, figures, references	5	
9	Viva-Voce	Understanding of the platform, design decisions and low-code concepts	10	
	TOTAL		**100**	

Table E.2: Performance Level Descriptors

Level	% of Max. Marks	Descriptor
Excellent	90–100	Feature implemented completely, correctly and beyond the basic requirement; student explains design choices confidently.
Good	70–89	Feature implemented correctly with minor gaps; explanation adequate.
Satisfactory	50–69	Feature partially implemented or works only for the main path; limited explanation.
Needs Improvement	< 50	Feature missing, non-functional or not understood by the student.

Signature of Internal Examiner

Name: _____

Signature of External Examiner

Name: _____

APPENDIX F – VIVA-VOCE RECORD SHEET

(For Examiner Use Only)

Student Name	SUBHALAKSHMI M	**Register No.**	240701539
Department / Section	Computer Science and Engineering	**Date of Viva**	
Application URL Verified	Yes / No	**Demo Duration**	_____ min

Table F.1: Viva-Voce Questions and Responses

Q.No.	Question / Topic Asked	Unit	Response (E / G / S / NI)	Remarks
1				
2				
3				
4				
5				
6				
7				
8				

E – Excellent G – Good S – Satisfactory NI – Needs Improvement

Table F.2: Marks Summary

Component	Max. Marks	Marks Awarded
Criteria 1–8 (Appendix E)	90	
Viva-Voce	10	
TOTAL	**100**	

Examiners' Remarks:

INTERNAL EXAMINER

Name: _____

EXTERNAL EXAMINER

Name: _____